```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
from google.colab import files
uploaded = files.upload()
```

Choose files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving IMDB Dataset.csv.zip to IMDB Dataset.csv (1).zip
```

```
!unzip "IMDB Dataset.csv (1).zip"
df = pd.read_csv("IMDB Dataset.csv")

df.head()
```

```
Archive:  IMDB Dataset.csv (1).zip
  inflating: IMDB Dataset.csv
```

	review	sentiment
0	One of the other reviewers has mentioned that ...	positive
1	A wonderful little production. The...	positive
2	I thought this was a wonderful way to spend ti...	positive
3	Basically there's a family where a little boy ...	negative
4	Petter Mattei's "Love in the Time of Money" is...	positive

```
print("Shape of dataset:", df.shape)

df.info()
```

```
Shape of dataset: (50000, 2)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 2 columns):
 #   Column     Non-Null Count  Dtype 
---  ------     --------------  ----- 
 0   review     50000 non-null  object
 1   sentiment  50000 non-null  object
dtypes: object(2)
memory usage: 781.4+ KB
```

```
df.isnull().sum()
```

	0
review	0
sentiment	0

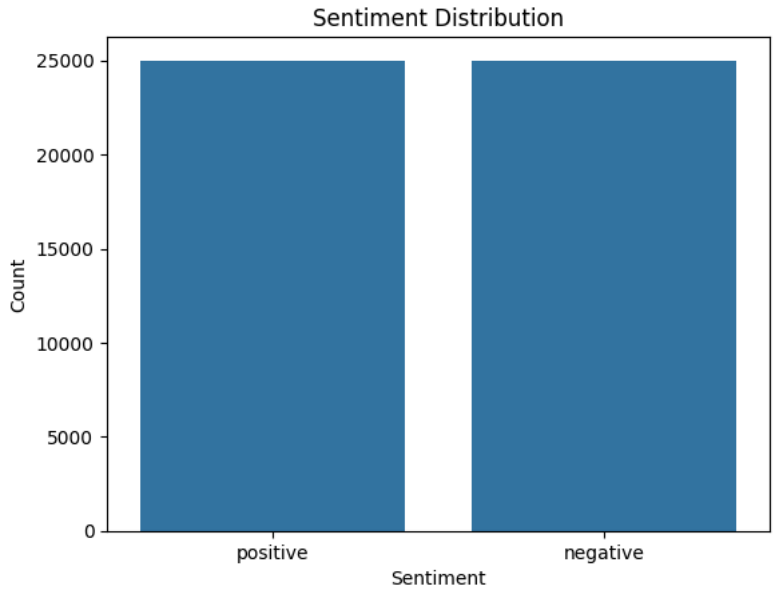
dtype: int64

```
df['sentiment'].value_counts()
```

sentiment	count
positive	25000
negative	25000

dtype: int64

```
sns.countplot(x='sentiment', data=df)

plt.title("Sentiment Distribution")
plt.xlabel("Sentiment")
plt.ylabel("Count")
plt.show()
```




```
df['sentiment'] = df['sentiment'].map({'positive':1, 'negative':0})

df.head()
```

	review	sentiment
0	One of the other reviewers has mentioned that ...	1
1	A wonderful little production. The...	1
2	I thought this was a wonderful way to spend ti...	1
3	Basically there's a family where a little boy ...	0
4	Petter Mattei's "Love in the Time of Money" is...	1

```
X = df['review']
y = df['sentiment']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
vectorizer = TfidfVectorizer(stop_words='english', max_features=5000)

X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)
```

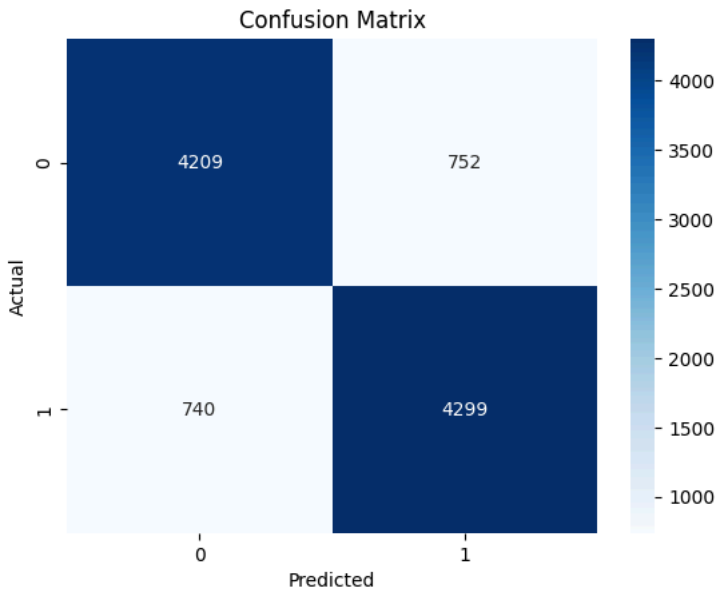
```
nb_model = MultinomialNB()

nb_model.fit(X_train_vec, y_train)

nb_pred = nb_model.predict(X_test_vec)
```

```
print("Naive Bayes Accuracy:", accuracy_score(y_test, nb_pred))

print(classification_report(y_test, nb_pred))
```

```
Naive Bayes Accuracy: 0.8508
              precision    recall  f1-score   support

           0       0.85      0.85      0.85      4961
           1       0.85      0.85      0.85      5039

    accuracy                           0.85     10000
   macro avg       0.85      0.85      0.85     10000
weighted avg       0.85      0.85      0.85     10000
```

```
cm = confusion_matrix(y_test, nb_pred)

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

plt.xlabel("Predicted")
plt.ylabel("Actual")
```

```
plt.title("Confusion Matrix")
plt.show()
```

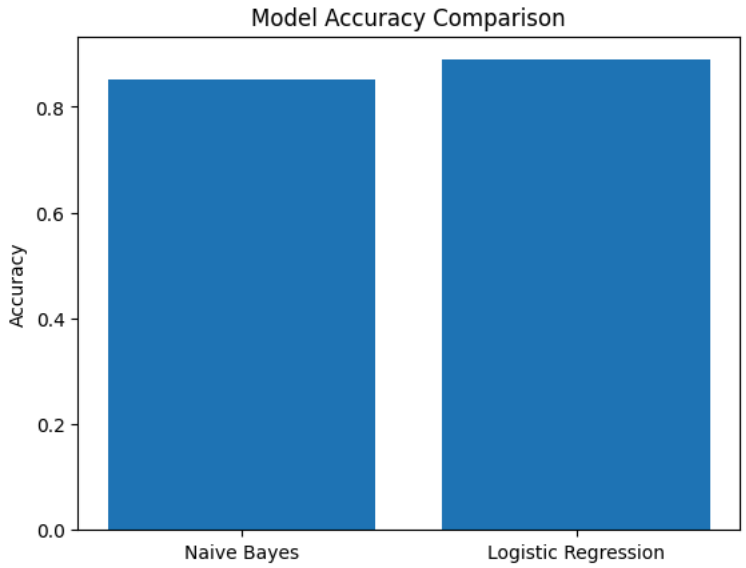



```
lr_model = LogisticRegression()

lr_model.fit(X_train_vec, y_train)

lr_pred = lr_model.predict(X_test_vec)
```

```
print("Logistic Regression Accuracy:", accuracy_score(y_test, lr_pred))

print(classification_report(y_test, lr_pred))
```

```
Logistic Regression Accuracy: 0.8889
              precision    recall  f1-score   support

           0       0.90      0.87      0.89      4961
           1       0.88      0.90      0.89      5039

    accuracy                           0.89     10000
   macro avg       0.89      0.89      0.89     10000
weighted avg       0.89      0.89      0.89     10000
```

```
models = ['Naive Bayes','Logistic Regression']
accuracy = [
    accuracy_score(y_test, nb_pred),
    accuracy_score(y_test, lr_pred)
]

plt.bar(models, accuracy)

plt.title("Model Accuracy Comparison")
plt.ylabel("Accuracy")
plt.show()
```




```
review = ["This movie was fantastic and the acting was great"]

review_vec = vectorizer.transform(review)

prediction = lr_model.predict(review_vec)

if prediction[0] == 1:
    print("Positive Review")
else:
    print("Negative Review")
```

```
Positive Review
```